

Rapport DevOps — DevOps Django Bank

Module : DevOps

Projet : Application bancaire avec chaîne CI/CD

Étudiant : El-Mehdi Taoufik

Année universitaire : 2025-2026

Dépôt GitHub : <https://github.com/Hasanpiker5543/devops>

1. Introduction

Ce rapport présente la réalisation d'un mini-projet DevOps basé sur une application web de gestion bancaire. Le projet ne se limite pas au développement applicatif : il intègre une démarche DevOps complète avec Git, Docker, Docker Compose, tests automatisés et pipeline CI/CD GitHub Actions.

L'objectif est de rendre l'application reproductible, testable et publiable automatiquement sous forme d'image Docker.

2. Présentation du sujet choisi

Le sujet choisi est une application bancaire. Elle permet à un administrateur de gérer les clients et les comptes bancaires, et à un client de consulter ses comptes, effectuer des transferts et suivre ses transactions.

3. Fonctionnalités de l'application

L'application contient les fonctionnalités principales attendues pour une application bancaire moderne.

- Page de connexion
- Page d'inscription
- Tableau de bord administrateur
- Tableau de bord client
- Gestion des clients
- Gestion des comptes bancaires
- Transfert d'argent
- Historique des transactions
- Export PDF

4. Architecture de l'application

L'architecture suit le modèle Django MVT : Model, View, Template. Les vues Django traitent les requêtes, les modèles représentent les données et les templates affichent les pages HTML.

```
Utilisateur -> Navigateur -> Django Views -> Templates  
Django Views -> Models -> SQLite
```

5. Choix techniques

Python et Django ont été choisis pour leur structure claire, leur système d'authentification intégré et leur productivité. SQLite est utilisé pour simplifier le déploiement local et la démonstration. Docker permet d'exécuter l'application dans un environnement reproductible.

6. Modèle de données

Le modèle de données repose sur trois entités principales : Client, Account et Transaction. Client est lié à l'utilisateur Django. Account représente un compte bancaire avec un solde. Transaction représente les opérations effectuées sur un compte.

7. Mise en œuvre

Le dossier banking contient la logique métier : modèles, formulaires, vues, routes, templates et tests. Le dossier bank_project contient la configuration globale du projet Django.

```
devops/  
  bank/  
    banking/  
    bank_project/  
    manage.py  
  Dockerfile  
  docker-compose.yml  
  requirements.txt  
  .github/workflows/ci-cd.yml
```

8. Stratégie Git

Le projet est versionné avec Git et publié sur GitHub. La branche main contient la version stable. Pour un workflow complet, une branche develop et des branches feature/* peuvent être utilisées avec des pull requests.

9. Conteneurisation Docker

Un Dockerfile a été ajouté afin de construire une image Docker de l'application. L'image installe les dépendances, copie le code, exécute les migrations, collecte les fichiers statiques et lance l'application avec Gunicorn.

10. Docker Compose

Le fichier docker-compose.yml décrit la stack complète. Il expose le service web sur le port 8000 et utilise un volume Docker pour conserver les données SQLite dans le dossier data.

```
docker compose up --build
```

11. Tests automatisés

Des tests Django ont été ajoutés pour vérifier la création des modèles Client et Account ainsi que l'accès à la page de connexion. Ces tests sont exécutables en ligne de commande.

```
cd bank  
python manage.py test
```

12. Pipeline CI/CD

Le pipeline GitHub Actions est déclenché à chaque push et pull request vers main ou develop. Il installe les dépendances, exécute les vérifications Django, lance les tests, construit l'image Docker et publie l'image sur GitHub Container Registry après un push réussi sur main.

- Installation des dépendances Python
- python manage.py check
- python manage.py test
- docker build
- publication GHCR sur main

13. Publication de l'image Docker

L'image Docker est publiée automatiquement sur GitHub Container Registry avec le tag `ghcr.io/hasanpiker5543/devops-django-bank:latest`.

14. Difficultés rencontrées

Une difficulté rencontrée concernait la publication Docker : le nom du propriétaire GitHub contenait des majuscules, alors que les tags Docker doivent être en minuscules. La solution a été d'utiliser explicitement `ghcr.io/hasanpiker5543/devops-django-bank:latest`.

15. Répartition du travail

Le projet a été réalisé par El-Mehdi Taoufik : développement de l'application, configuration Docker, création du pipeline CI/CD, rédaction du README et préparation du rapport.

16. Conclusion

Ce projet répond aux objectifs du module DevOps : une application fonctionnelle est versionnée, testée, conteneurisée et intégrée dans une chaîne CI/CD. La solution facilite la reproduction de l'environnement, l'exécution des tests et la publication automatique de l'image Docker.

17. Perspectives

Des améliorations possibles incluent l'ajout de PostgreSQL dans Docker Compose, davantage de tests unitaires et d'intégration, un environnement de staging, et éventuellement des manifestes Kubernetes pour le bonus.

18. Annexes

```
python manage.py migrate
python manage.py runserver
docker compose up --build
```

Lien dépôt : <https://github.com/Hasanpiker5543/devops>

Lien pipeline : <https://github.com/Hasanpiker5543/devops/actions/workflows/ci-cd.yml>